Fully Convolutional Networks for Semantic Segmentation

Jonathan Long Evan Shelhamer Trevor Darrell – CVPR 2015

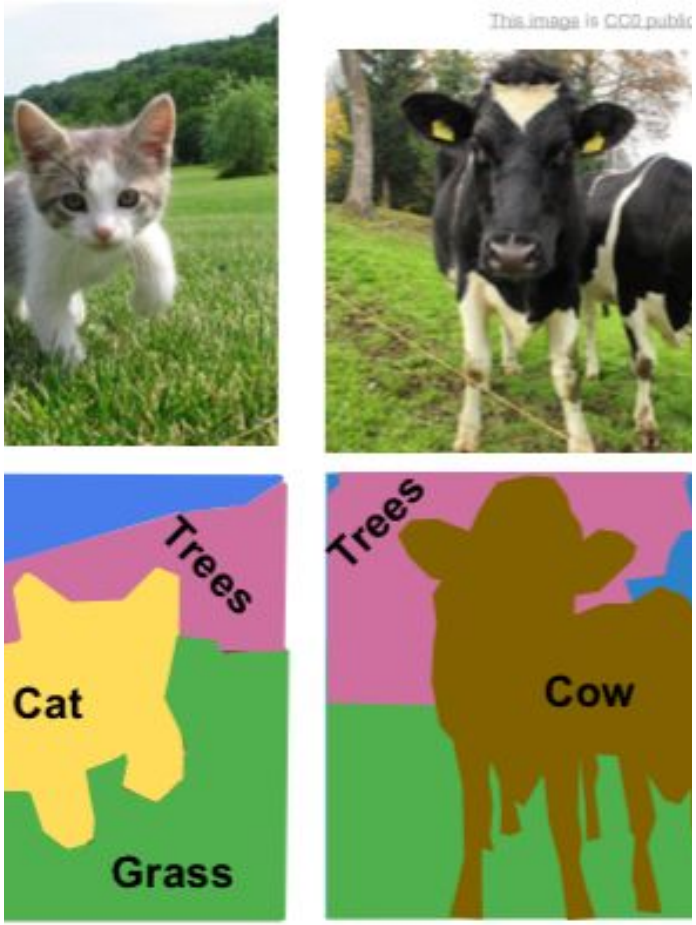
Soma Biswas
Department of Electrical Engineering,
Indian Institute of Science, Bangalore.

Slide Credit: Stanford Course: Convolutional Neural Network for Visual Recognition

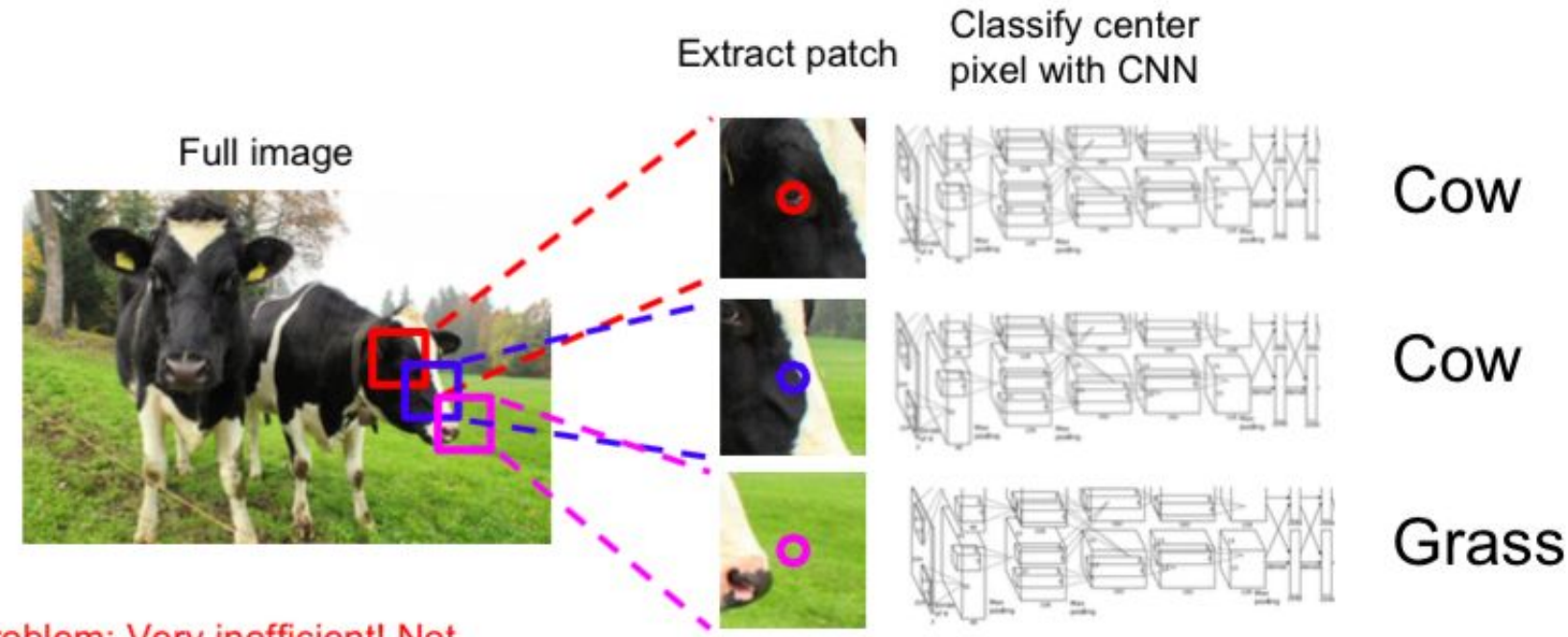
Semantic Segmentation

Label each pixel in the image with a category label

Don't differentiate instances, only care about pixels



Approach 1

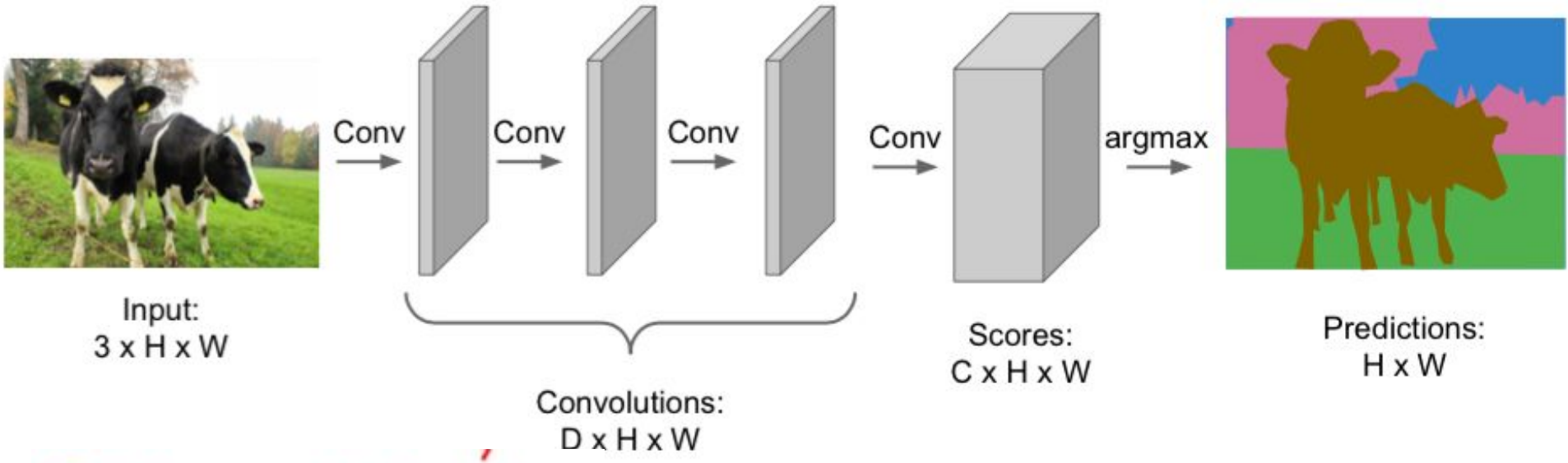


Problem: Very inefficient! Not reusing shared features between overlapping patches

Farabet et al, "Learning Hierarchical Features for Scene Labeling," TPAMI 2013
Pinheiro and Collobert, "Recurrent Convolutional Neural Networks for Scene Labeling", ICML 2014

Approach 2

Design a network as a bunch of convolutional layers to make predictions for pixels all at once!

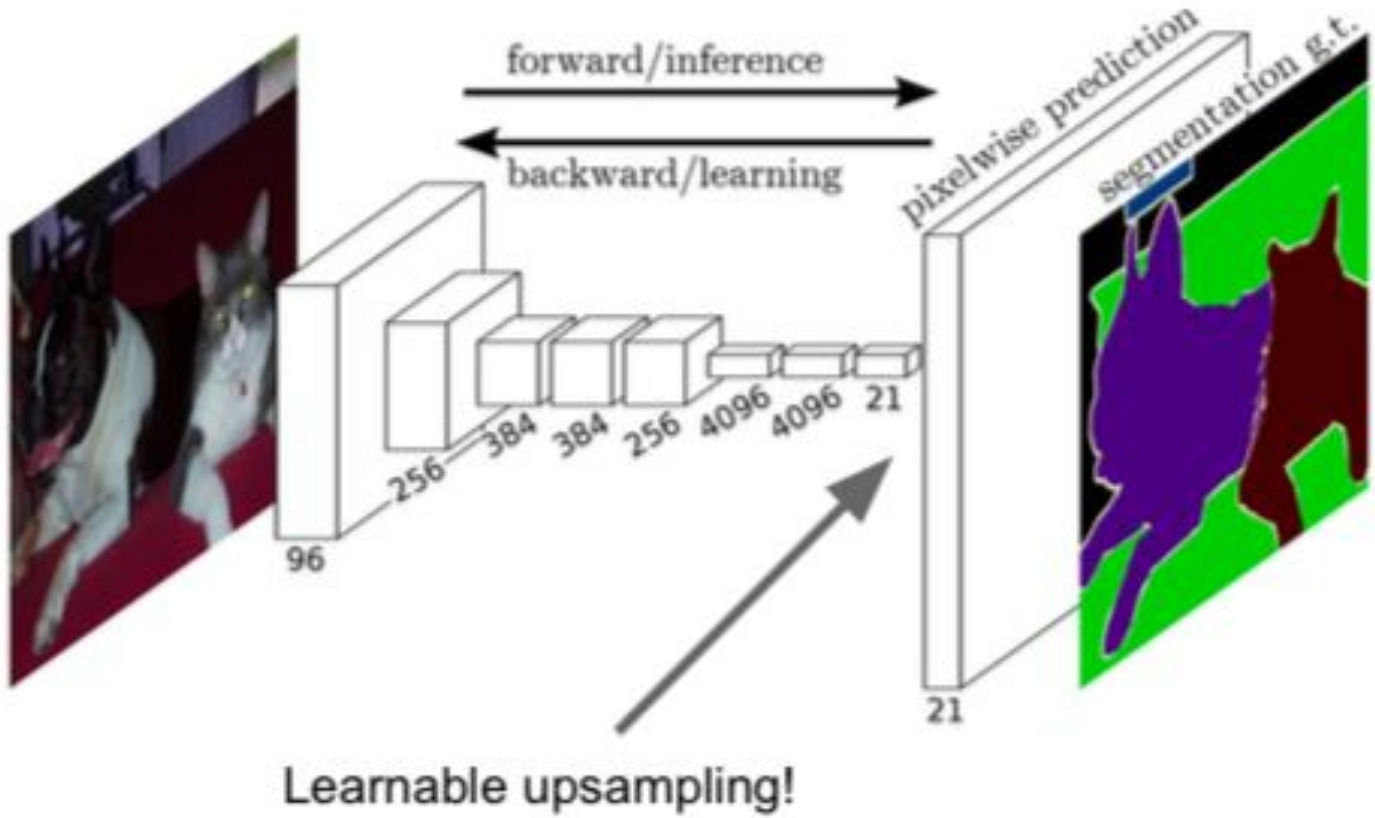


Problem: convolutions at original image resolution will be very expensive ...

From paper!

ing layer. We decapitate each net by discarding the final classifier layer, and convert all fully connected layers to convolutions. We append a 1×1 convolution with channel dimension 21 to predict scores for each of the PASCAL classes (including background) at each of the coarse output locations, followed by a deconvolution layer to bilinearly upsample the coarse outputs to pixel-dense outputs as described in Section 3.3. Table 1 compares the prelim-

Better Idea!



Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015

In-Network upsampling: “Unpooling”

Nearest Neighbor

1	2
3	4

Input: 2 x 2

→

1	1	2	2
1	1	2	2
3	3	4	4
3	3	4	4

Output: 4 x 4

“Bed of Nails”

1	2
3	4

Input: 2 x 2

→

1	0	2	0
0	0	0	0
3	0	4	0
0	0	0	0

Output: 4 x 4

In-Network upsampling: “Max Unpooling”

Fei-Fei Li & Justin Johnso

Max Pooling
Remember which element was max!

1	2	6	3
3	5	2	1
1	2	2	1
7	3	4	8

Input: 4 x 4

→

5	6
7	8

Output: 2 x 2

→ . . . →
Rest of the network

Max Unpooling
Use positions from pooling layer

1	2
3	4

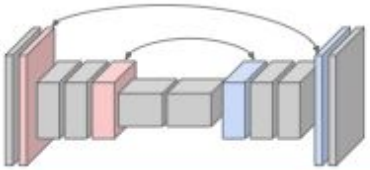
Input: 2 x 2

→

0	0	2	0
0	1	0	0
0	0	0	0
3	0	0	4

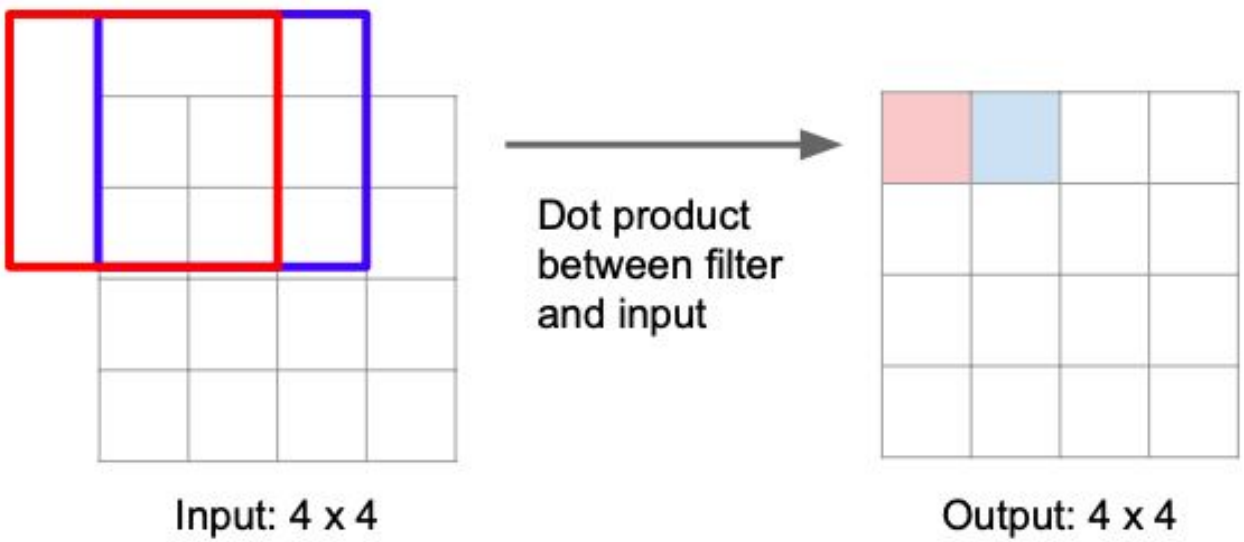
Output: 4 x 4

Corresponding pairs of downsampling and upsampling layers



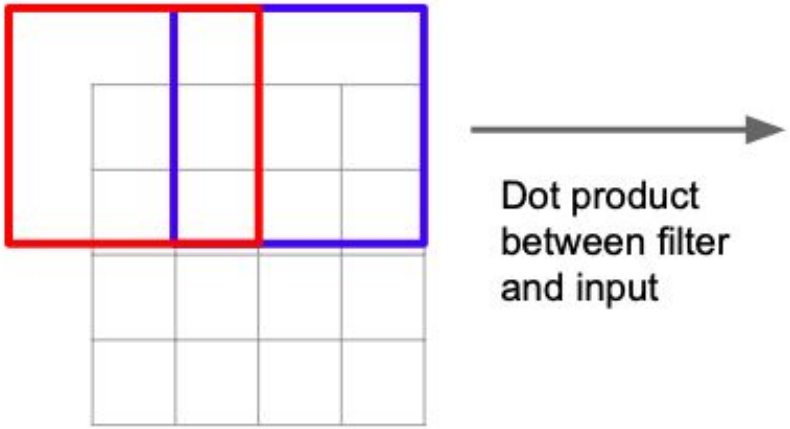
Learnable Upsampling: Transpose Convolution

Recall: Normal 3 x 3 convolution, stride 1 pad 1



Learnable Upsampling: Transpose Convolution

Recall: Normal 3 x 3 convolution, stride 2 pad 1



Dot product between filter and input

Input: 4 x 4

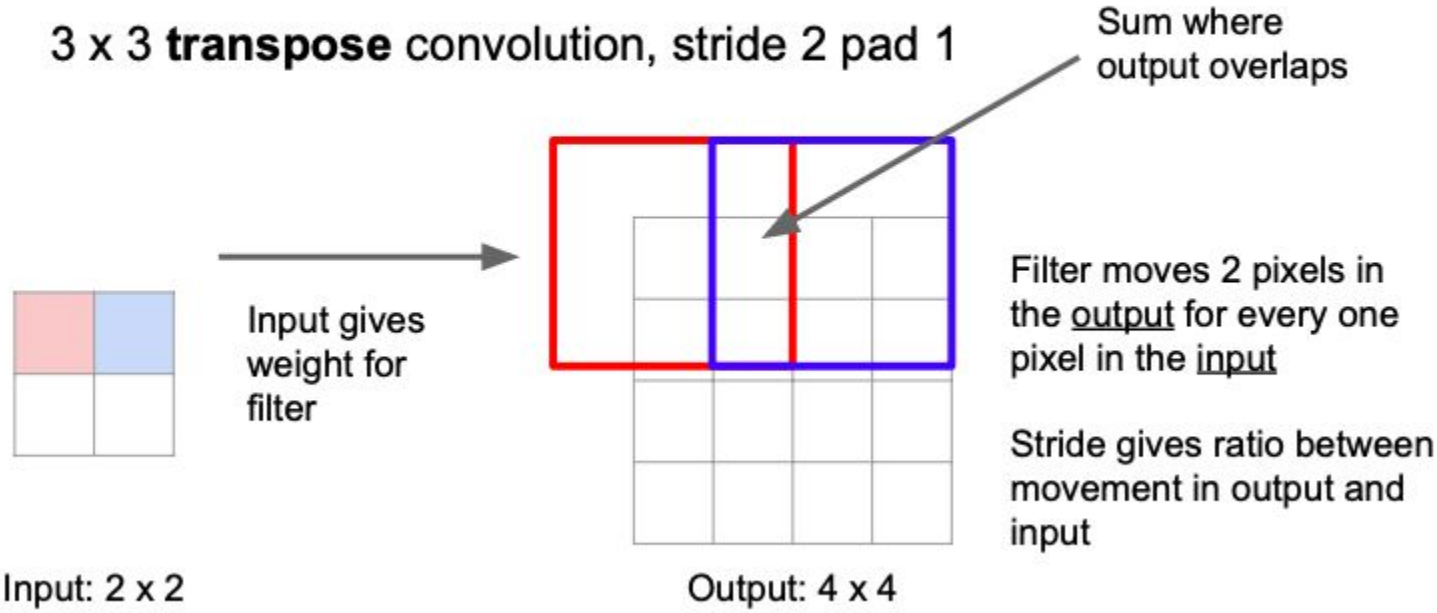
Output: 2 x 2

Filter moves 2 pixels in the input for every one pixel in the output

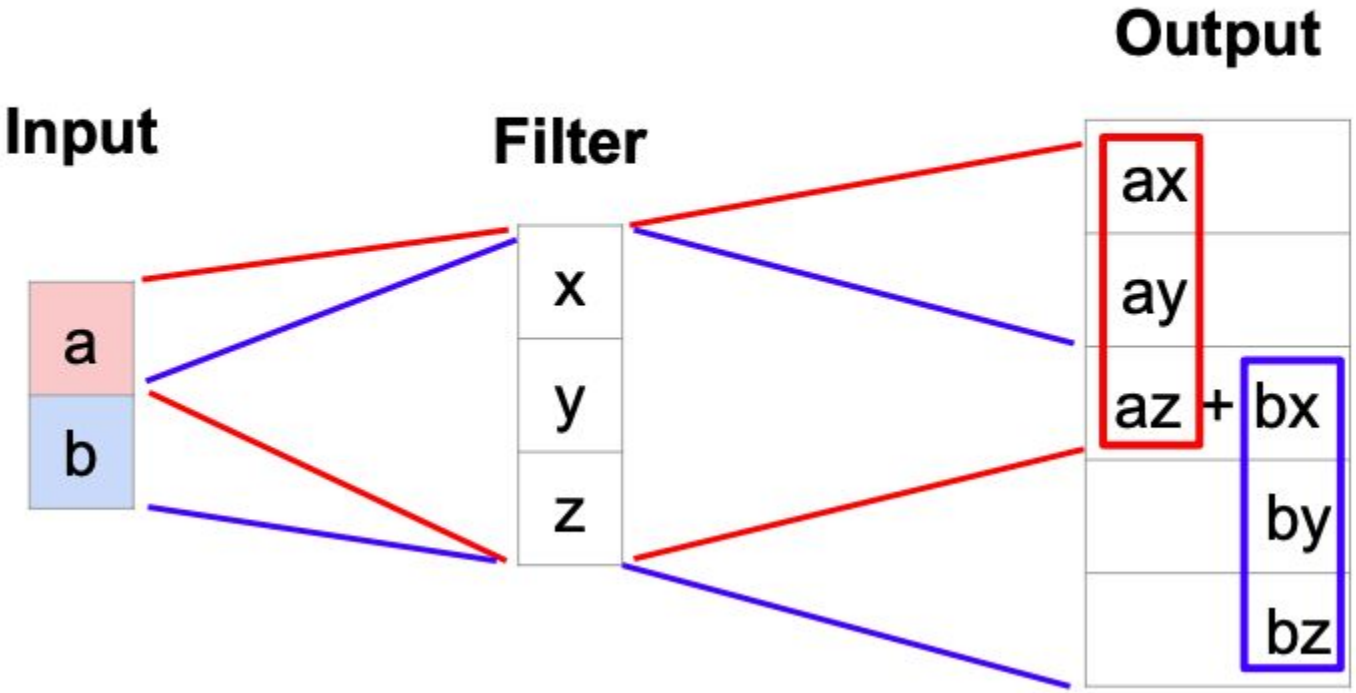
Stride gives ratio between movement in input and output

Learnable Upsampling: Transpose Convolution

3 x 3 **transpose** convolution, stride 2 pad 1



Transpose Convolution: 1D Example



Output contains copies of the filter weighted by the input, summing at where at overlaps in the output

Need to crop one pixel from output to make output exactly 2x input

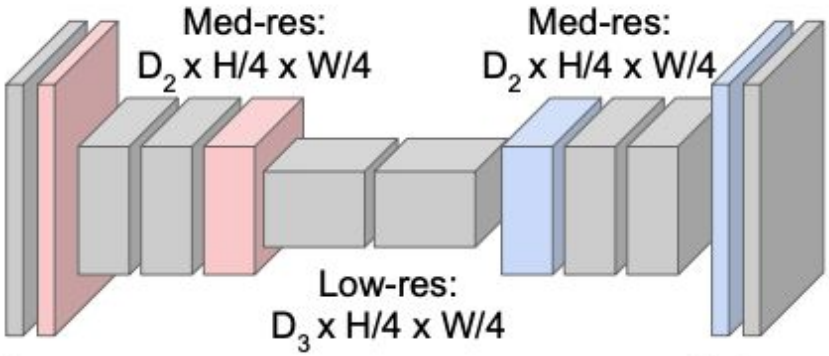
Semantic Segmentation Idea: Fully Convolutional

Downsampling: Pooling, strided convolution

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!

Upsampling: Unpooling or strided transpose convolution

Input:
3 x H x W



High-res:
D_1 x H/2 x W/2

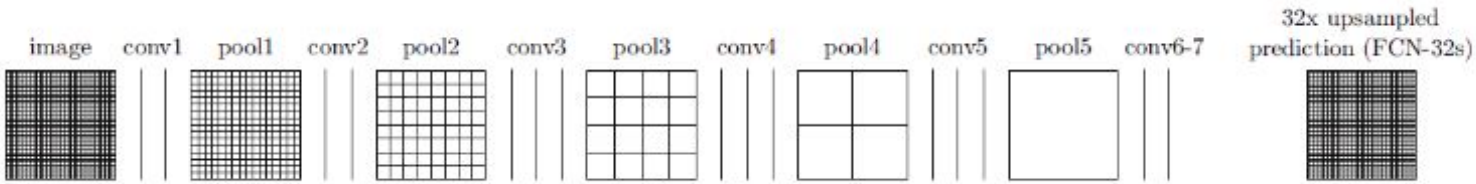
High-res:
D_1 x H/2 x W/2

Predictions:
H x W

.ong, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015
Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

Fusing the output

After going through conv7 as below, the output size is small, then 32× upsampling is done to make the output have the same size of input image. But it also makes the output label map rough. And it is called **FCN-32s**:

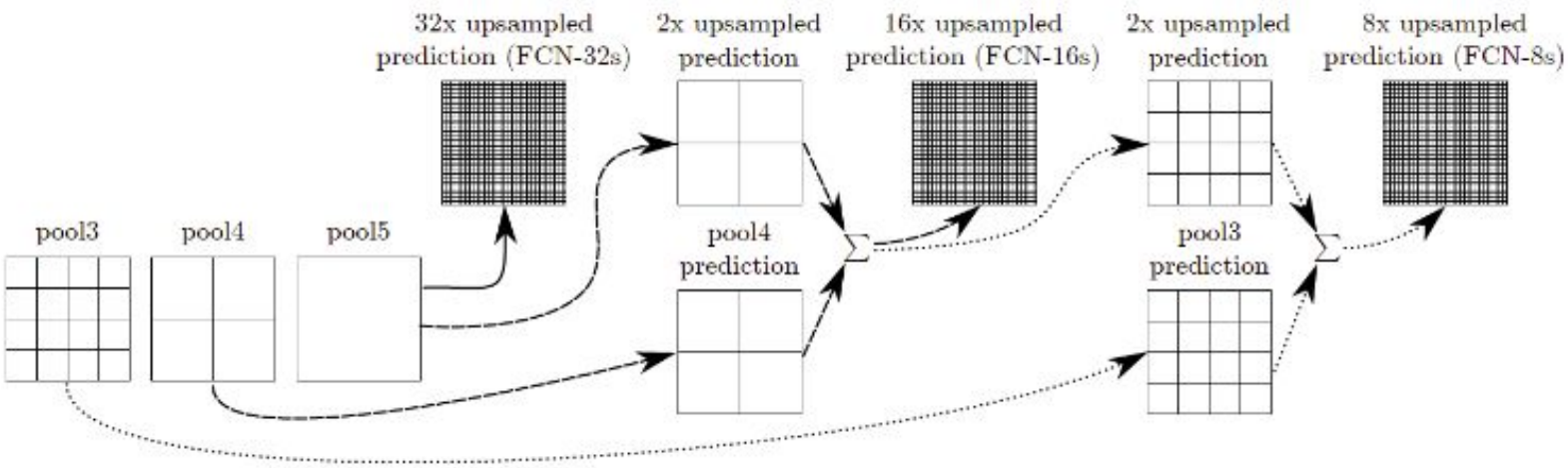


FCN-32s

This is because, **deep features can be obtained when going deeper, spatial location information is also lost when going deeper.** That means output from shallower layers have more location information. If we combine both, we can enhance the result.

Fusion

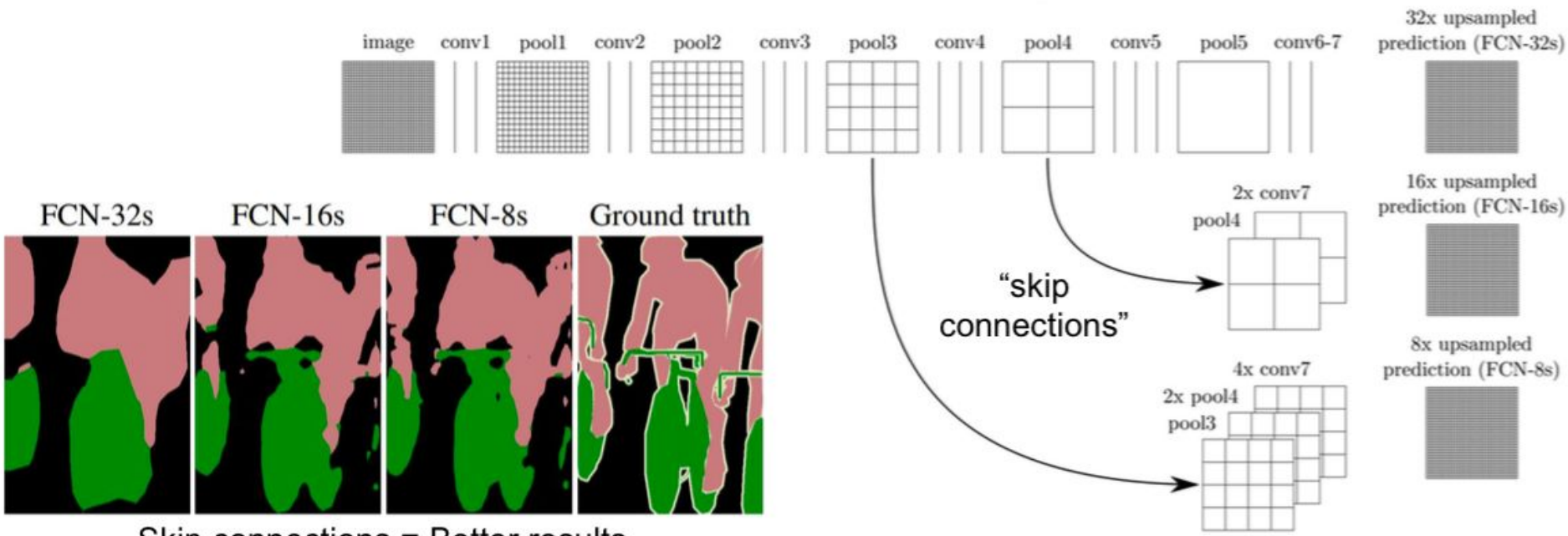
To combine, we **fuse the output (by element-wise addition)**:



Fusing for FCN-16s and FCN-8s

FCN-16s: The output from pool5 is 2× upsampled and fuse with pool4 and perform 16× upsampling. Similar operations for **FCN-8s** as in the figure above.

Upsampling

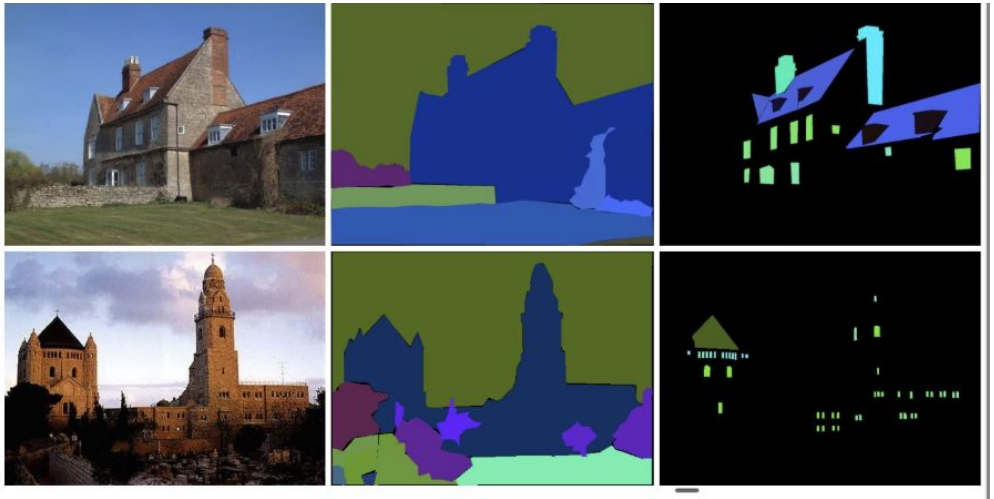


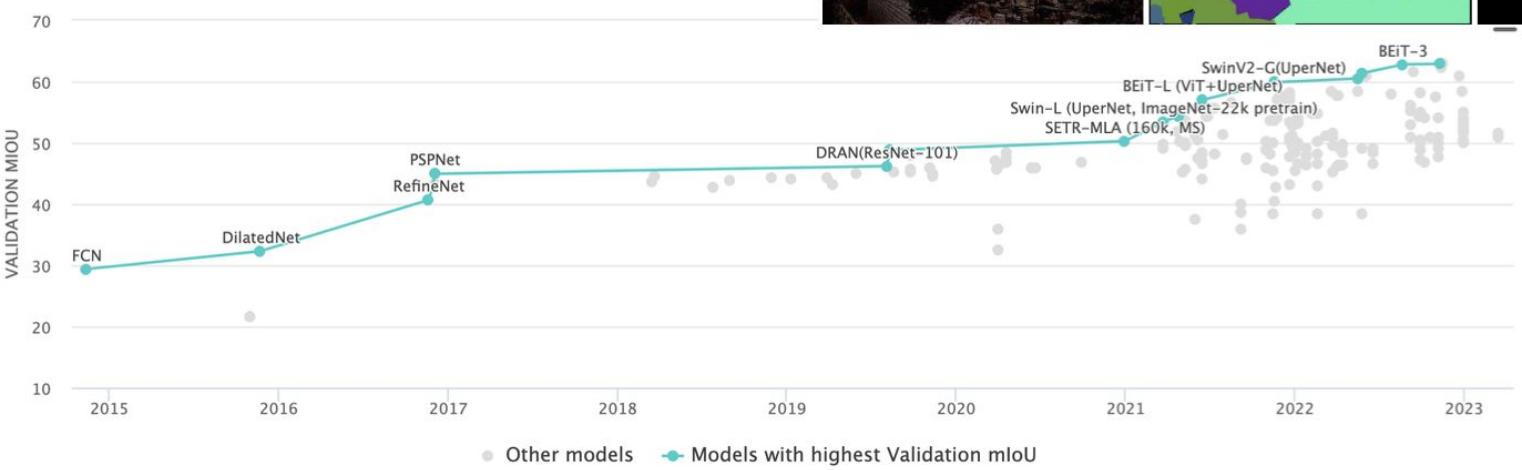
Skip connections = Better results

Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015

Example: Semantic Segmentation Dataset: ADE20K

- 20K scene-centric images exhaustively annotated with pixel-level objects and object parts labels.
- 150 semantic categories, which include stuffs like sky, road, grass, and discrete objects like person, car, bed.





Evaluation Metric

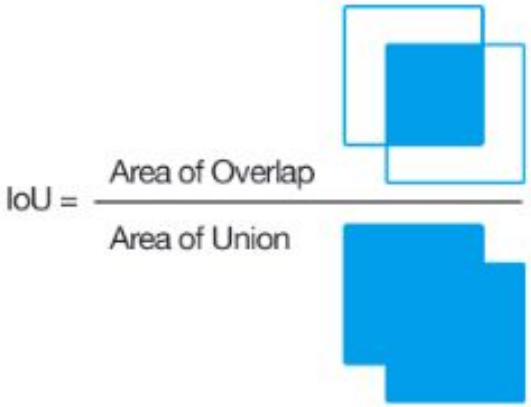
Per pixel accuracy

This metric is self explanatory, since it outputs the class prediction accuracy per pixel.

$$acc(P, GT) = \frac{|\text{pixels correctly predicted}|}{|\text{total nb of pixels}|} \tag{1}$$

Jaccard (Intersection over Union)

This evaluation metric is often used for image segmentation, since it is more structured. The jaccard is a per class evaluation metric, which computes the number of pixels in the intersection between the predicted and ground truth segmentation maps for a given class, divided by the number of pixels in the union between those two segmentation maps, also for that given class.

$$jacc(P(class), GT(class)) = \frac{|P(class) \cap GT(class)|}{|P(class) \cup GT(class)|} \tag{2}$$

where P is the predicted segmentation map and GT is the ground truth segmentation map. $P(class)$ is then the binary mask indicating if each pixel is predicted as *class* or not. In general, the closer to 1, the better.